



A CSR initiative by



kotak
Kotak Mahindra Bank

Kotak IISc AI-ML Centre



TANUH

Translational Artificial Intelligence
for Networked Universal Healthcare

Deep Foundation Models

Lecture Series - Part 2

Sriram Ganapathy

Electrical Engineering, IISc.

Scope

- ✓ In this lecture series - what is covered
 - * The question of “what” in the topics listed
 - * A part of the “why” as well
 - * And a small part of “where” it is used
- ✓ In this lecture series - what is not covered
 - * Mathematical derivations
 - * Details of various methods, analysis and understanding
 - * Alternate approaches (I provide only a biased sampling)
 - * Implementation aspects

Contents - This Lecture

- ✓ 2.1 Large language models
 - * Tokenizers and sampling
- ✓ 2.2 Multimodal alignment
- ✓ 2.3 Pre-training and post-training
- ✓ 2.4 Reinforcement learning for fine-tuning
 - * Proximal policy optimization
 - * Direct policy optimization

Contents - This Lecture

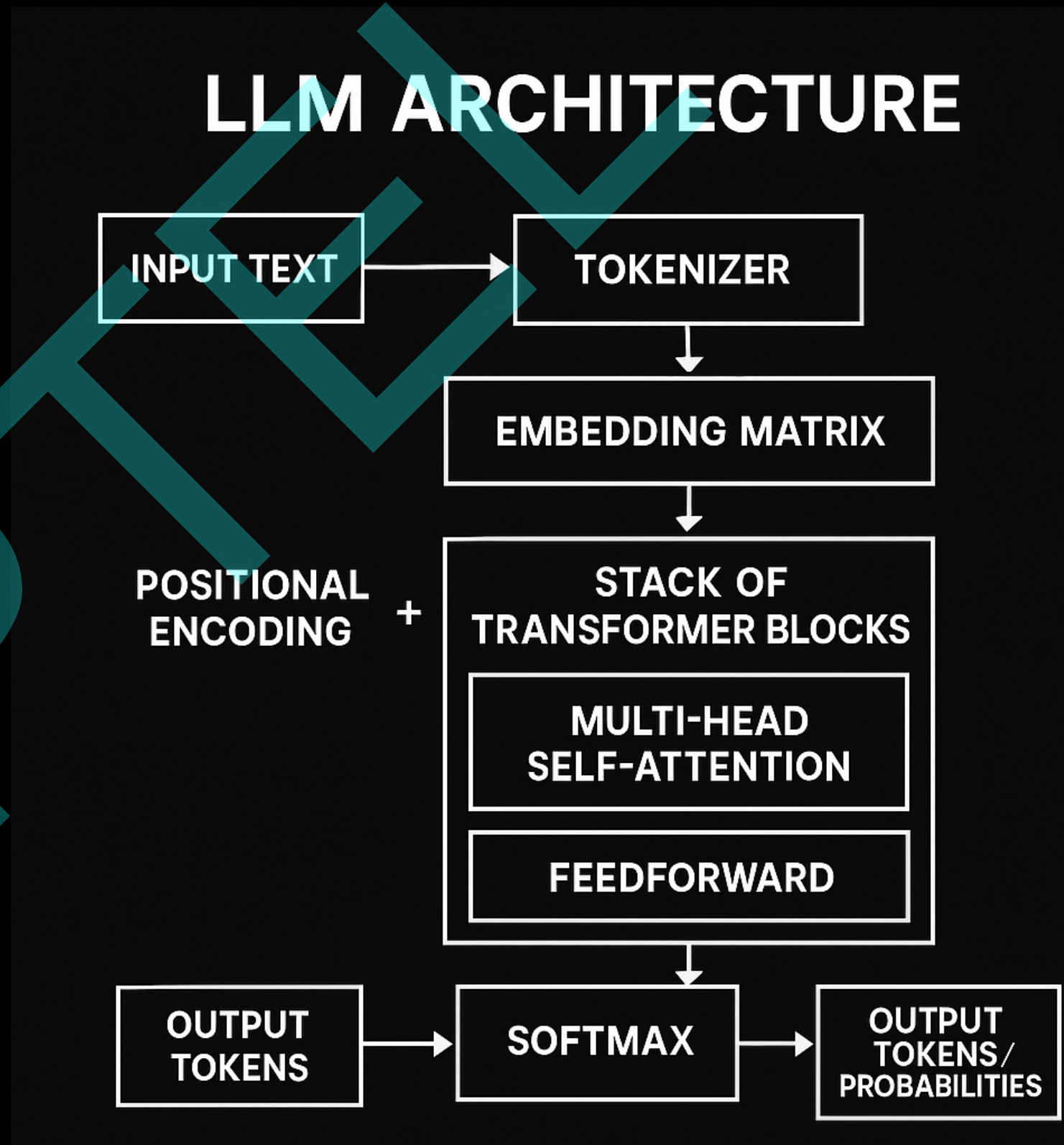
- ✓ 2.1 Large language models
 - * Tokenizers and sampling
- ✓ 2.2 Multimodal alignment
- ✓ 2.3 Pre-training and post-training
- ✓ 2.4 Reinforcement learning for fine-tuning
 - * Proximal policy optimization
 - * Direct policy optimization

2.1 Large language models

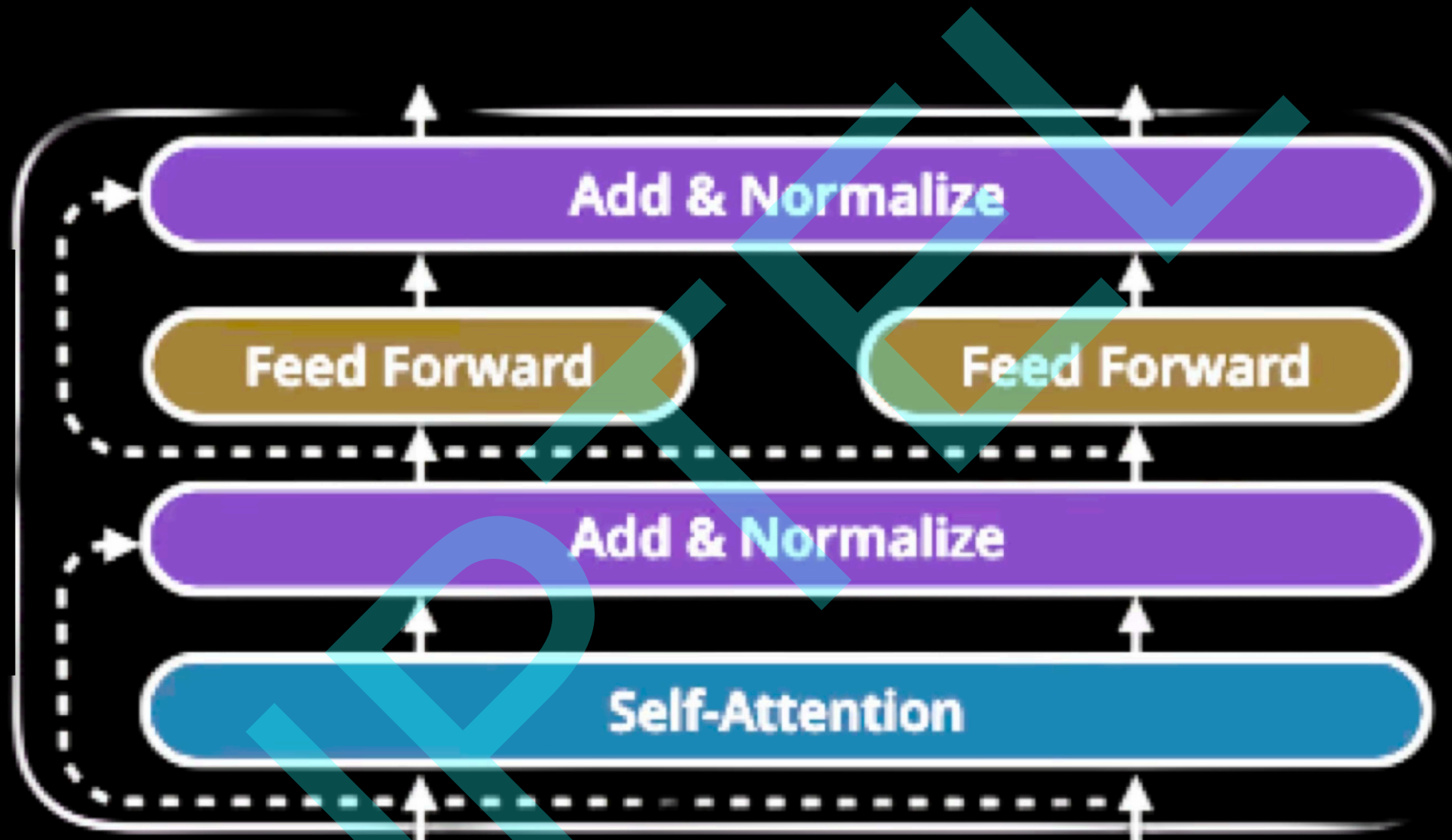
Large Language Models (LLMs) are **AI systems trained on massive amounts of text** to understand, generate, and reason with human language. They are built using **deep neural networks**, usually with the **Transformer** architecture, and can perform a wide variety of tasks without task-specific training.

LLM - Big picture

- ✓ Builds upon the transformer architecture.
- ✓ Input text/multi-modal data is converted to tokens - **Tokenizer**
 - *An embedding matrix maps the large vocabulary of the token outputs to real-valued vector inputs.
- ✓ Following the stack of transformers, linear + **soft-max** layer generates the output tokens.
 - *Derive the outputs using **sampling** techniques



2.1 Large Language Models



✓ LLMs are decoder only transformer models trained with a next token prediction task

● Language modeling task

Inputs — Tokenizer

- ✓ We have been using word “token” (What is it)
 - * A token is a basic unit of text that a language model reads and processes. It is not the same as a word—it can be:
 - A whole word
 - A sub-word (partial word)
 - Punctuations and special symbols

“I am a student of machine learning.”

Tokenizer

- ✓ Breaking text into tokens allows the model to:
 - * Work with a consistent vocabulary
 - * Handle rare words by splitting them
- ✓ Compress common phrases into fewer tokens
 - * Typically in English 1 Token is 4-5 characters
 - * Performed in a data-driven fashion
 - ◎ Using variable length segmentation
 - Most common method is byte-pair encoding (BPE)

Tokenizer

A New Algorithm for Data Compression

Philip Gage

1994

Tokenizer training - BPE

- ✓ Goal: Represent text using a small vocabulary of subword units so that:
- ✓ Frequent words become single tokens or a few tokens
 - * Rare or unseen words can still be represented as sequences of subwords
 - * The overall sequence length (in tokens) is reduced compared to pure character-level
- ✓ BPE learns these subword units automatically from a corpus.
 - * Start with:
 - A training corpus (lots of text)
 - Initial vocabulary = individual characters (plus maybe a special end-of-word marker)
 - Then iteratively merge the most common adjacent symbol pair.

Tokenizer — BPE

✓ Example

*low → l o w </w> (4 tokens)

*lower → l o w e r </w> (6 tokens)

*lowest → l o w e s t </w> (7 tokens)

*slow → s l o w </w> (5 tokens)

*slower → s l o w e r </w> (7 tokens)

✓ Total tokens - 29

Tokenizer — BPE

✓ Example

- * low → l o w </w> (4 tokens)
- * lower → l o w e r </w> (6 tokens)
- * lowest → l o w e s t </w> (7 tokens)
- * slow → s l o w </w> (5 tokens)
- * slower → s l o w e r </w> (7 tokens)

✓ Total tokens - 29

✓ Algorithm - Step 1

- * First merge: ('l', 'o') → "lo"
- * As ('l', 'o') (appears 5 times)
- * Token counts now:
 - * lo w </w> → 3
 - * lo w e r </w> → 5
 - * lo w e s t </w> → 6
 - * s lo w </w> → 4
 - * s lo w e r </w> → 6
- * Total = 24 tokens

Tokenizer — BPE

✓ Example

- * low → l o w </w> (4 tokens)
- * lower → l o w e r </w> (6 tokens)
- * lowest → l o w e s t </w> (7 tokens)
- * slow → s l o w </w> (5 tokens)
- * slower → s l o w e r </w> (7 tokens)

✓ Total tokens - 29

✓ Algorithm - Step 2

- * Second merge: ('lo', 'w') → "low"
- * As ('lo', 'w') (appears 5 times)

* Tokens now

- * low </w> → 2
- * lower </w> → 4
- * lowest </w> → 5
- * slow </w> → 3
- * slower </w> → 5
- * Total = 19 tokens

Tokenizer — BPE

✓ Example

- * low → l o w </w> (4 tokens)
- * lower → l o w e r </w> (6 tokens)
- * lowest → l o w e s t </w> (7 tokens)
- * slow → s l o w </w> (5 tokens)
- * slower → s l o w e r </w> (7 tokens)

✓ Total tokens - 29

✓ Algorithm - Step 3

- * Add token lowe and merge:
- * Token counts:
 - * low </w> → 2
 - * lowe r </w> → 3
 - * lowe s t </w> → 4
 - * s low </w> → 3
 - * s lowe r </w> → 4
- * Total = 16 tokens

Same process can be repeated on any sequence of discrete inputs

Output generation from LLM

- ✓ Generating outputs from LLMs is called decoding.
- ✓ Let us start with an example

Name an animal which is both domestic and wild

Token	Probability
"cat"	0.60
"dog"	0.25
"car"	0.10
"quantum"	0.05

If we choose the **argmax** token at each step, you get the same response every time.

To get multiple different responses, resort to **sampling**

Output generation - sampling

✓ Let the input be denoted as x and output be denoted as y

✓ Greedy decoding

$$* y_t = \operatorname{argmax}_i p(y_t = i | y_{<t}, x)$$

✓ Sampling

* From a multinomial distribution

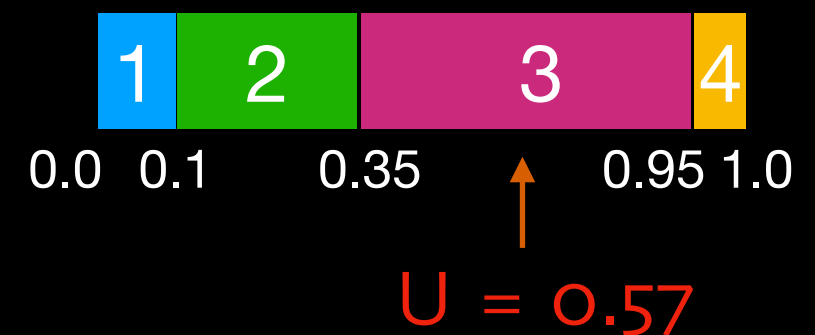
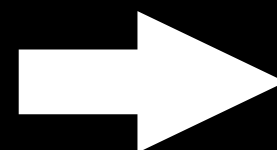
* In LLMs these are the soft-max outputs.

Output generation - sampling

- ✓ Sampling from a multinomial distribution — simplified
 - * Pick a random number U from a uniform random number generator $[0, 1.0)$. Say $U = 0.57$
 - * Convert the LLM outputs into cumulative probabilities

Token	Probability
"car" [Indx 1]	0.10
"dog" [Indx 2]	0.25
"cat" [Indx 3]	0.60
"quantum" [Indx 4]	0.05

Example output



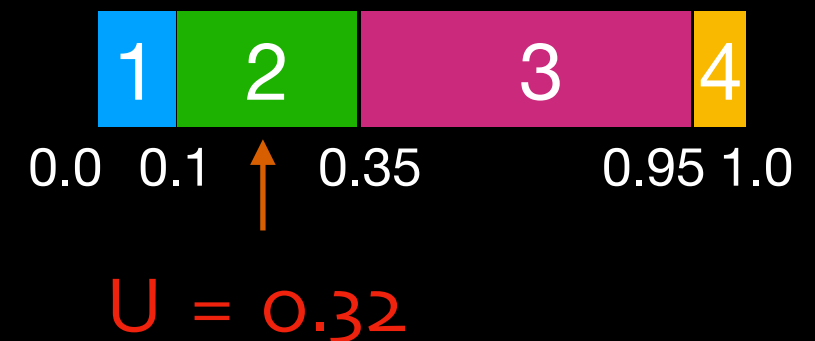
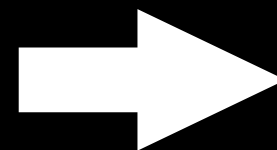
Output will be "car"

Output generation - sampling

- ✓ Sampling from a multinomial distribution — simplified
 - * Pick a random number U from a uniform random number generator $[0, 1.0)$ with another seed. Say $U = 0.32$
 - * Convert the LLM outputs into cumulative probabilities

Token	Probability
"car" [Indx 1]	0.10
"dog" [Indx 2]	0.25
"cat" [Indx 3]	0.60
"quantum" [Indx 4]	0.05

Example output



Output will be "dog"

Output generation - sampling

✓ Top-K sampling

- ⦿ Keeping only the top-K (pre-logits)
- ⦿ Only apply soft-max on those token indices
- ⦿ Perform a sampling using K-multinomial distribution.

✓ Top-p nucleus sampling

- ⦿ Keep all plausible outputs till the total probability achieves a pre-set value.
 - For example, if $p = 0.8$, then all tokens are selected starting from the top-1, until the cumulative probability is ≥ 0.8 . i.e. ("cat", "dog") get selected.
 - Renormalize the probabilities
 - Perform multinomial sampling using the outputs obtained

Output generation - sampling

✓ Temperature sampling $p(i) = \frac{\exp \frac{z_i}{T}}{\sum_j \exp \frac{z_j}{T}}$ where T is the temperature.

● T=1: normal softmax.

● T<1: distribution becomes sharper (more peaked).

● T>1: distribution becomes flatter (more uniform).

- Use the modified distribution and perform multinomial sampling

Output generation - temperature sampling

✓ Example (pre-softmax outputs)

✓ "cat": 5

✓ "dog": 4

✓ "car": 1

At $T = 1.0$

$p \propto [e^5, e^4, e^1] \approx [148.4, 54.6, 2.7]$



At $T = 2$

$p \propto [e^{2.5}, e^2, e^{0.5}] \approx [12.2, 7.4, 1.6]$



✓ Intuitive effect on behavior

✓ With high temperature:

* More randomness / diversity

* The model explores more unusual words and phrases.

✓ Factual correctness tends to drop.

✓ The model may contradict itself more.

✓ Long-range coherence can suffer.

✓ More creative but also more chaotic

✓ Good for brainstorming, story generation, poetry, etc.

✓ Bad for code, math, or precise instructions.

Summary thus far ...

✓ Inputs to LLMs

* Text (or multimodal) tokens

- Derived in a data-driven fashion from a corpus
- Fixed (but large) vocabulary size (of the order of 10k-100k-1M)
- Embedding matrix (learnable) converts them to input representations for the transformer stack.

✓ Outputs from the LLMs

* Greedy decoding versus sampling

- Multiple approaches to sampling - temperature, top-K, top-p. Or a combination of these.

Time for some ...

NPTFEL

Q&A

Contents - This Lecture

✓ 2.1 Large language models

- * Tokenizers

✓ 2.2 Multimodal alignment

✓ 2.3 Pre-training and post-training

✓ 2.4 Reinforcement learning for fine-tuning

- * Proximal policy optimization

2.2 LLM pre-training

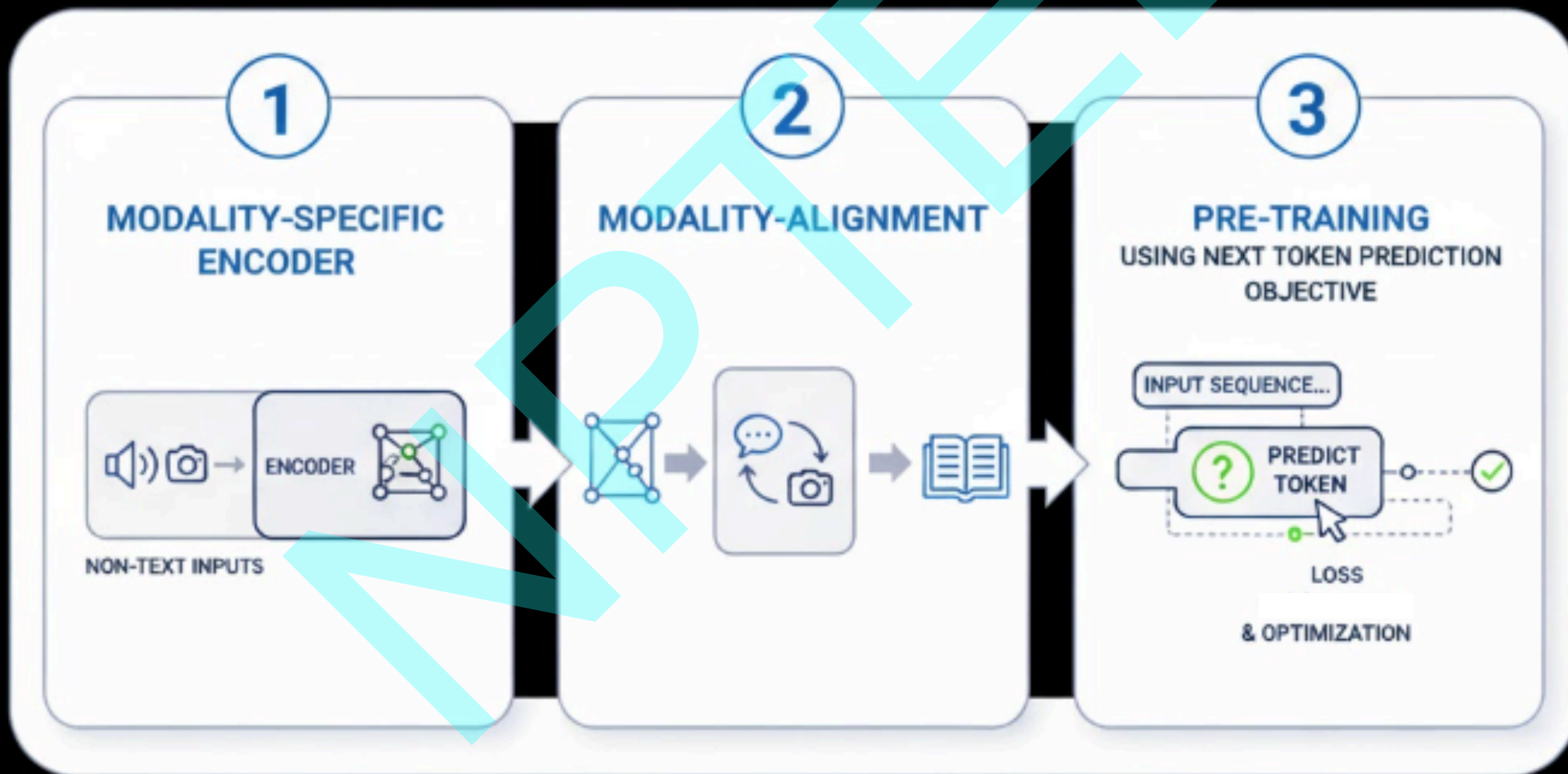
✓ Multi-modal LLMs

* Using mixture of text, audio, speech, images and video data

● Can also include other data (finance, biomedical data, etc ...)

2.2 LLM pre-training

KEY STEPS



Pre-training — Modality encoder

✓ For each non-text modality:

- * Images → Vision transformer (ViT / SigLIP / EVA) → visual tokens
- * Audio → EnCodec, BEATS or spectrogram encoder → acoustic tokens
- * Video → Space-time ViT tokens
- * 3D → Point encoder / voxel encoder
- * Speech → Whisper-like encoder
- * Structured data → embedding table

Contents - This Lecture

- ✓ 2.1 Large language models
 - * Tokenizers
- ✓ 2.2 Multimodal alignment
- ✓ 2.3 Pre-training and post-training
- ✓ 2.4 Reinforcement learning for fine-tuning
 - * Proximal policy optimization

Pre-training objective

* Model is learned to predict the next output prediction in a sequence

$$\mathcal{L} = - \sum_t P(\mathbf{x}_t | \mathbf{x}_{<t})$$

- ◎ Most of the language modeling and creative generation tasks can be represented as a slot-filling task
 - For example, question answering can be considered as structured next token prediction of the form (<Question>, ...)
 - Can be extended to multi-modal data too.

2.2 LLM post-training

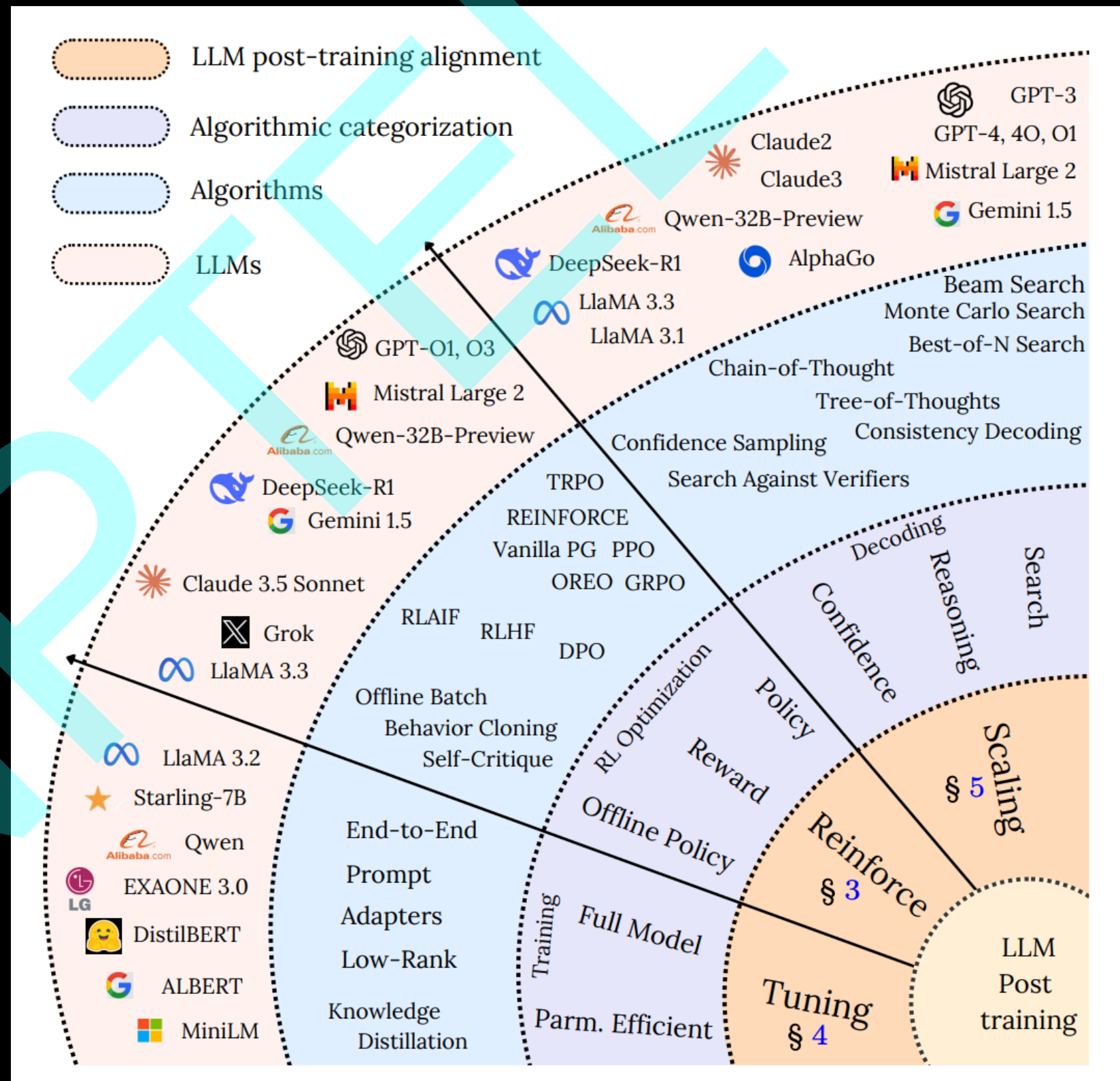
- ✓ While pre-training provides a broad linguistic foundation, post-training methods enable LLMs to refine their knowledge, improve reasoning, enhance factual accuracy, and align more effectively with user intents and ethical considerations.
- ✓ Fine-tuning, reinforcement learning, and test-time scaling

LLM Post-Training: A Deep Dive into Reasoning Large Language Models

Komal Kumar*, Tajamul Ashraf*, Omkar Thawakar, Rao Muhammad Anwer, Hisham Cholakkal, Mubarak Shah, Ming-Hsuan Yang, Phillip H.S. Torr, Fahad Shahbaz Khan, Salman Khan

Taxonomy of LLM post-training

- ✓ Significant efforts have been undertaken to post-training.
- ✓ Broadly categorized as tuning, reinforcement learning and test-time scaling.
- ✓ Plethora of publicly released models fall under one of these buckets.



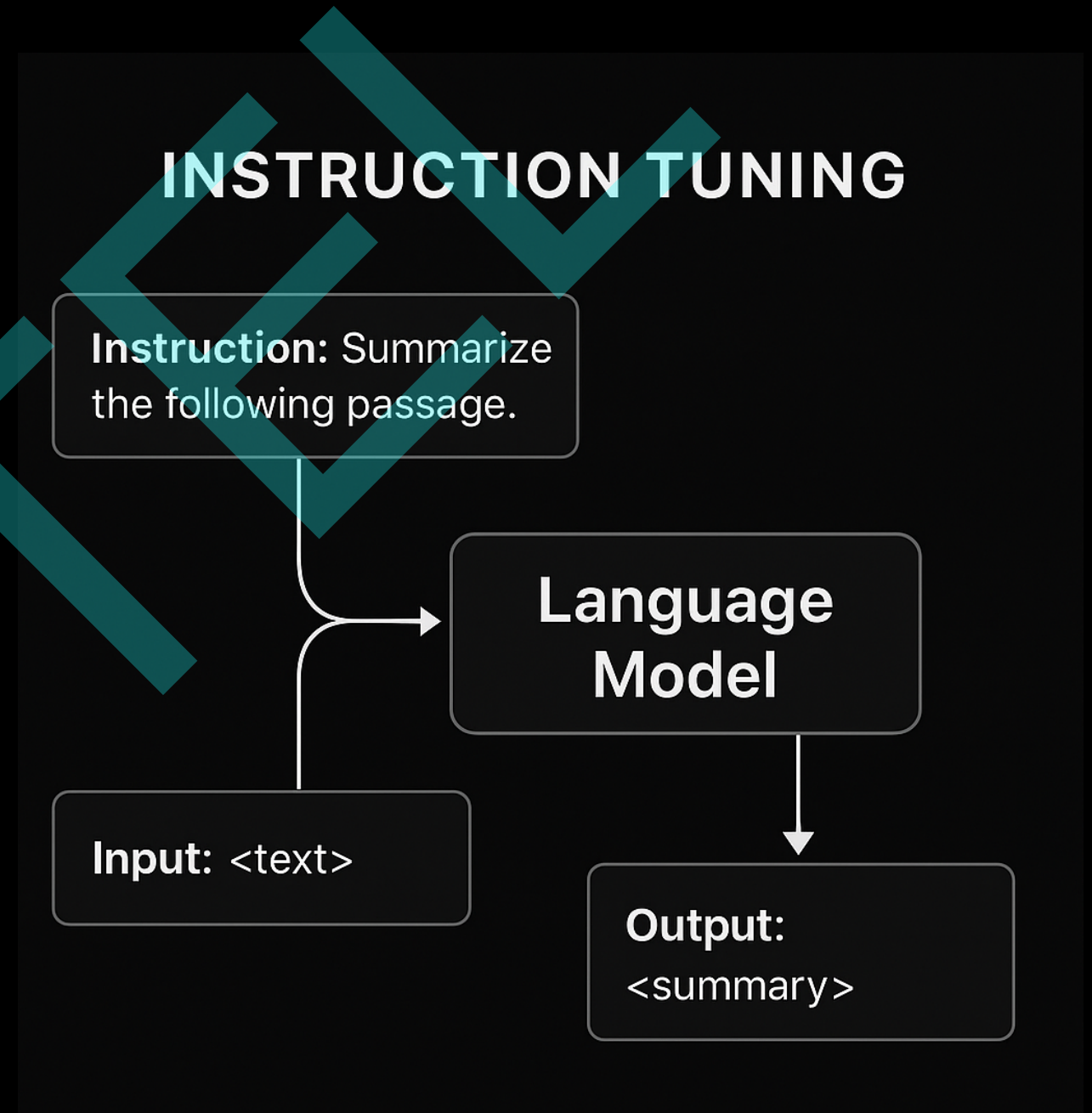
2.2 LLM post-training

✓ Fine-tuning

- * In pre-training models are trained on vast datasets, but without enforcing any structure in the input data.
- * Fine-tuning enhances performance in tasks like sentiment analysis, question answering, and domain-specific applications such as medical diagnosis.
- * Fine-tuning is supervised, however can suffer from over-fitting or memorization.
 - Typically handled by parameter efficient fine-tuning like LoRA.
- * Fine-tuning can also reduce generalization to new tasks

LLM - Instruction tuning

- ✓ Provide detailed instructions to the model.
- ✓ Fine-tune on multiple tasks without making it specific to one-single task.
- ✓ Evaluate on seen and un-seen tasks during inference.



Instruction Tuning

FINETUNED LANGUAGE MODELS ARE ZERO-SHOT LEARNERS

Jason Wei*, Maarten Bosma*, Vincent Y. Zhao*, Kelvin Guu*, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le

Google Research

- ✓ Taking a pre-trained language model and fine-tune on many downstream tasks
- * Provide detailed instructions to execute the tasks.
- * A method called Fine-tuned language net (FLAN)

FLAN Instruction tuning

Finetune on many tasks (“instruction-tuning”)

Input (Commonsense Reasoning)

Here is a goal: Get a cool sleep on summer days.

How would you accomplish this goal?

OPTIONS:

-Keep stack of pillow cases in fridge.

-Keep stack of pillow cases in oven.

Target

keep stack of pillow cases in fridge

Input (Translation)

Translate this sentence to Spanish:

The new office building was built in less than three months.

Target

El nuevo edificio de oficinas se construyó en tres meses.

Sentiment analysis tasks

Coreference resolution tasks

...

Inference on unseen task type

Input (Natural Language Inference)

Premise: At my age you will probably have learnt one lesson.

Hypothesis: It's not certain how many lessons you'll learn by your thirties.

Does the premise entail the hypothesis?

OPTIONS:

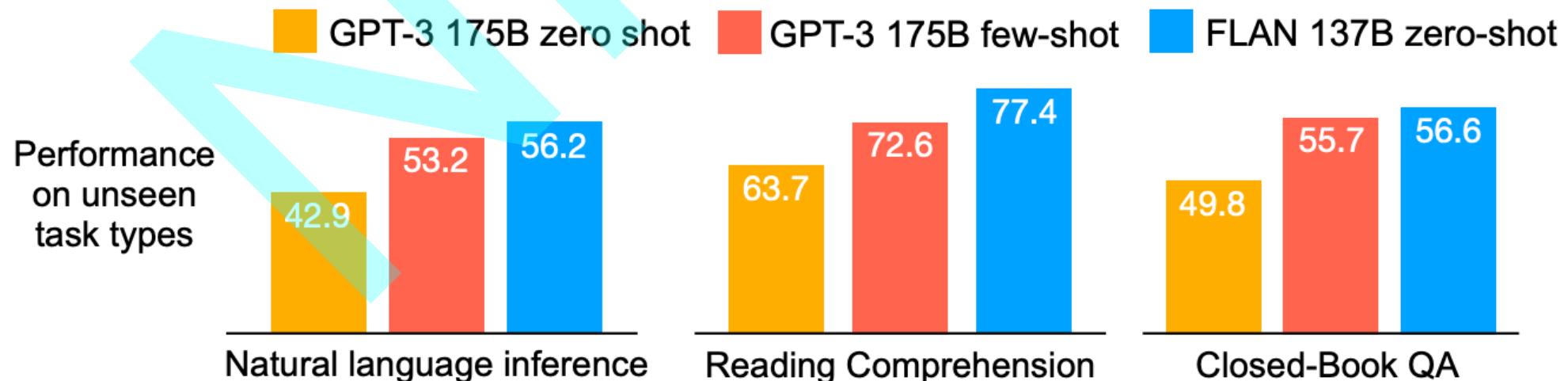
-yes

-it is not possible to tell

-no

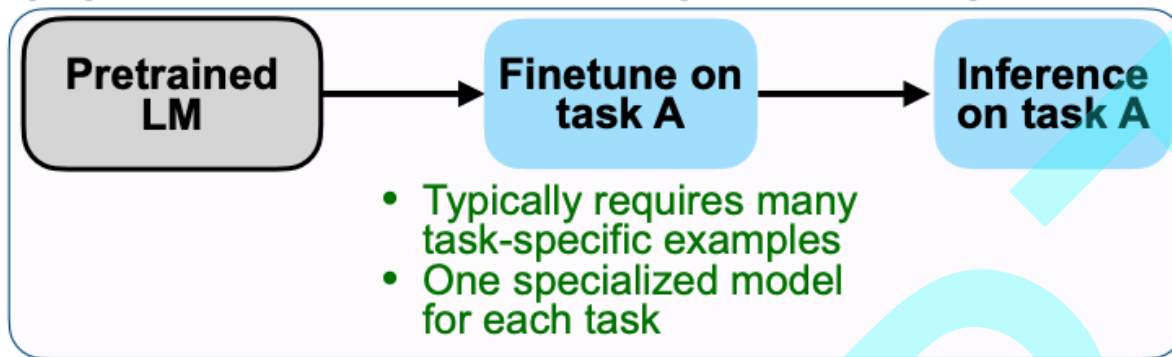
FLAN Response

It is not possible to tell

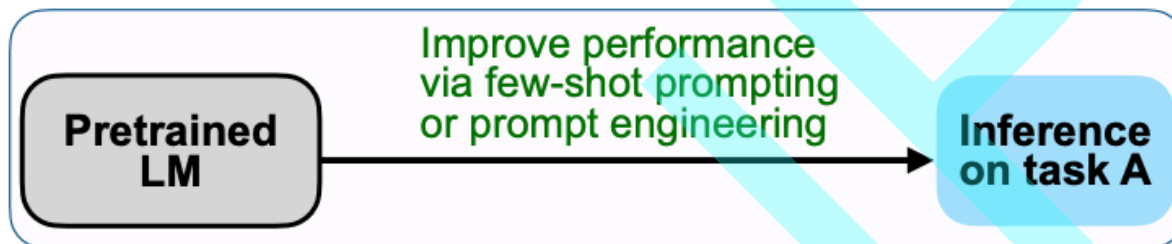


Instruction tuning versus fine-tuning

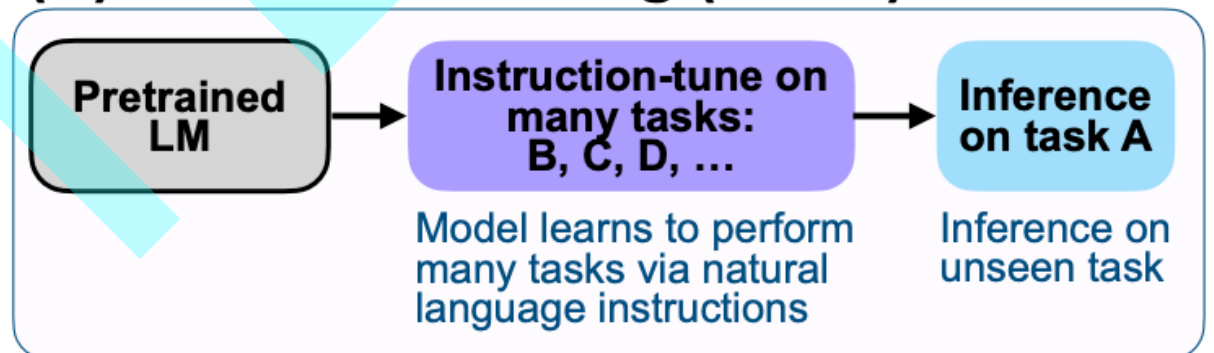
(A) Pretrain–finetune (BERT, T5)



(B) Prompting (GPT-3)



(C) Instruction tuning (FLAN)



Instruction tuning versus fine-tuning

Aspect	Fine-Tuning	Instruction Tuning
Goal	Optimize one task	Make model follow instructions across tasks
Data	Task-specific	Multi-task, natural-language instructions
Behavior	Specialist	Generalist assistant
Output style	Rigid, task-specific	Conversational, flexible
Typical use	Benchmark tasks	Chatbots, assistants

Time for some ...

NPTFEL

Q&A

Contents - This Lecture

- ✓ 2.1 Large language models
 - * Tokenizers and sampling
- ✓ 2.2 Multimodal alignment
- ✓ 2.3 Pre-training and post-training
- ✓ 2.4 **Reinforcement learning for fine-tuning**
 - * **Proximal policy optimization**
 - * **Direct policy optimization**

2.4 Reinforcement learning

✓ Reinforcement learning

- * Long-term reward/punishment losses

- * a framework where an agent interacts with an environment over discrete time steps.

- Let s_t denote state of the system at time t

- Let a_t denote the action performed by the system at time t

- The agent aims to learn a policy $\pi(a | s)$ that maximizes the

cumulative reward $\sum_{t=0}^T r^t$

- The environment also returns the next state s_{t+1}

Why RL is needed for LLMs

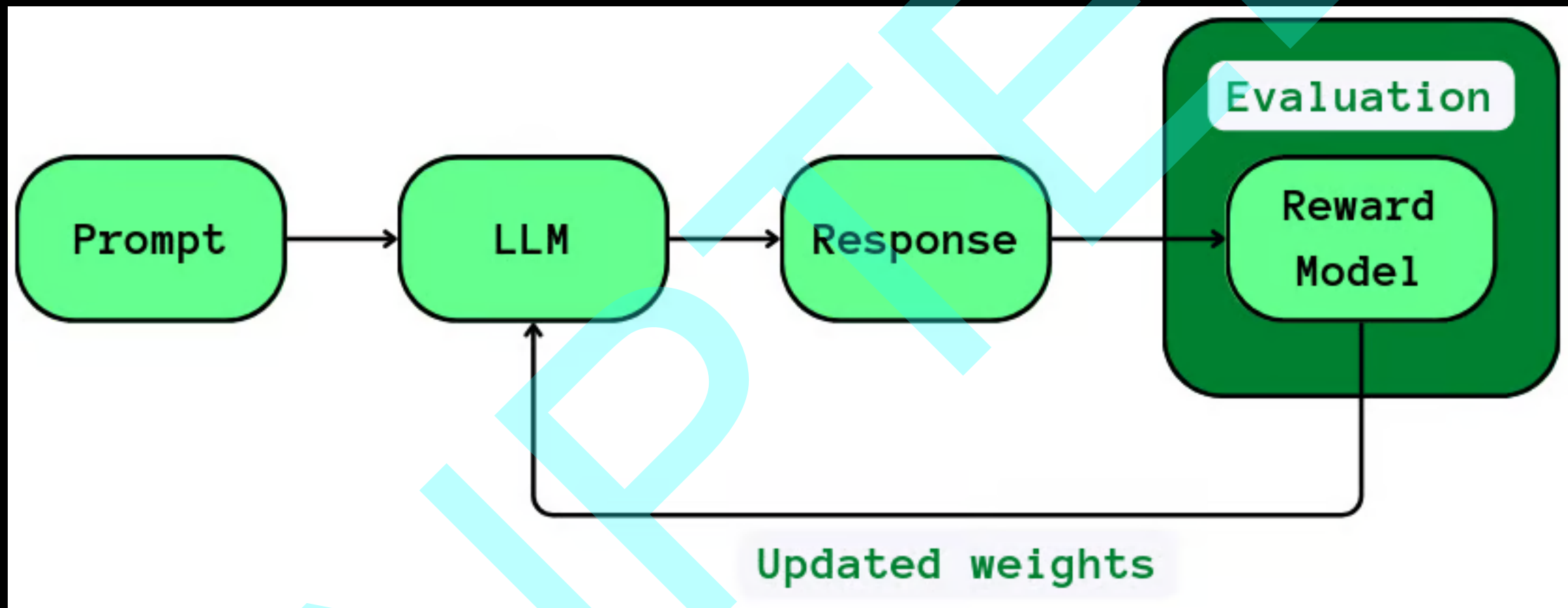
✓ Supervised fine-tuning (SFT) teaches:

- ⦿ what humans would say
- ⦿ by mimicking curated or hard-labeled outputs

✓ RL teaches:

- ⦿ what humans would prefer
- ⦿ what is safe or aligned
 - by maximizing a reward function that reflects human judgments
 - This reward can encode:
 - helpfulness, harmlessness and truthfulness
 - style preferences, reduction of hallucination

RL in LLMs - Big Picture



RL in LLMs

✓ In the RL perspective:

* State (s) → the prompt or conversation history

* Action (a) → the generated token(s)

* Policy (π_{θ}) → the LLM parameterized by θ

* Reward (R) → scalar score from a human or a learned reward model

✓ Trajectory (τ) → full generated sequence

$$\tau = (a_1, a_2, \dots, a_T)$$

✓ LLMs are autoregressive policies:

$$* \pi_{\theta}(a_t | s_t) = P_{\theta}(\text{token}_t | \text{context})$$

RL in LLMs - PPO

Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
`{joschu, filip, prafulla, alec, oleg}@openai.com`

RL with Proximal Policy Optimization

- ✓ One of the most common implementation of RL in LLMs
 - * Using proximal policy optimization (PPO)
 - * Preference data

Human Preference Data

Prompt

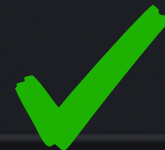
Response A

Response B

What's the best way to get rid of a cold?

You can try drinking plenty of fluids and getting lots of rest.

I think that exercising and staying active is most effective.



RL with Proximal Policy Optimization

- ✓ One of the most common implementation of RL in LLMs
 - * Using proximal policy optimization (PPO)
 - * Preference data

Human Preference Data		
Prompt	Response A	Response B
Explain climate change.	Climate change is the result of complex natural cycles.	Climate change is primarily due to human activities such as burning fossil fuels. ✓

RL with Proximal Policy Optimization

- ✓ Collect good amount of human preference data
 - * Train a reward model
 - * Given a pair of responses (y_1, y_2) for the same prompt x
 - * humans provide a preference:
 - For example, $y_1 > y_2$
 - * The reward model $r_\phi(x, y)$ is trained so that:
 - $r_\phi(x, y_1) - r_\phi(x, y_2)$ is maximized
 - i.e., reward model outputs a scalar $r_\phi(x, y)$
 - Such that $\mathcal{L}_{RM} = -\log[\sigma(r_\phi(x, y_1)) - \sigma(r_\phi(x, y_2))]$ is optimized

RL with Proximal Policy Optimization

✓ Once the reward model is trained, the LLM model is trained

✓ Denoting the LLM as $\pi_{\theta}(\cdot | x)$

$$J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} \left[r_{\phi}(x, y) - \beta D_{\text{KL}}(\pi_{\theta}(\cdot | x) \parallel \pi_{\text{SFT}}(\cdot | x)) \right]$$

* Here $\pi_{\text{SFT}}(\cdot | x)$ denotes the Supervised fine-tuned reference model.

● Included to reduce model-drift

● The contribution of the two losses are controlled by β

✓ Maximize $J(\theta)$ to update the LLM (θ) to improve the alignment with the human-preference data.

RL with Proximal Policy Optimization

✓ Let us consider an example

Prompt x: “Explain photosynthesis to a 7-year-old.”

Pair	Prompt	Response	Human preference
1	x	A: “Photosynthesis is the biochemical process whereby chlorophyll molecules in plant chloroplasts convert photons into chemical energy via the Calvin cycle.”	✗ (rejected)
1	x	B: “Plants use sunlight, water, and air to make their own food. It’s like them cooking a meal using sunshine as the stove.”	✓ (chosen)

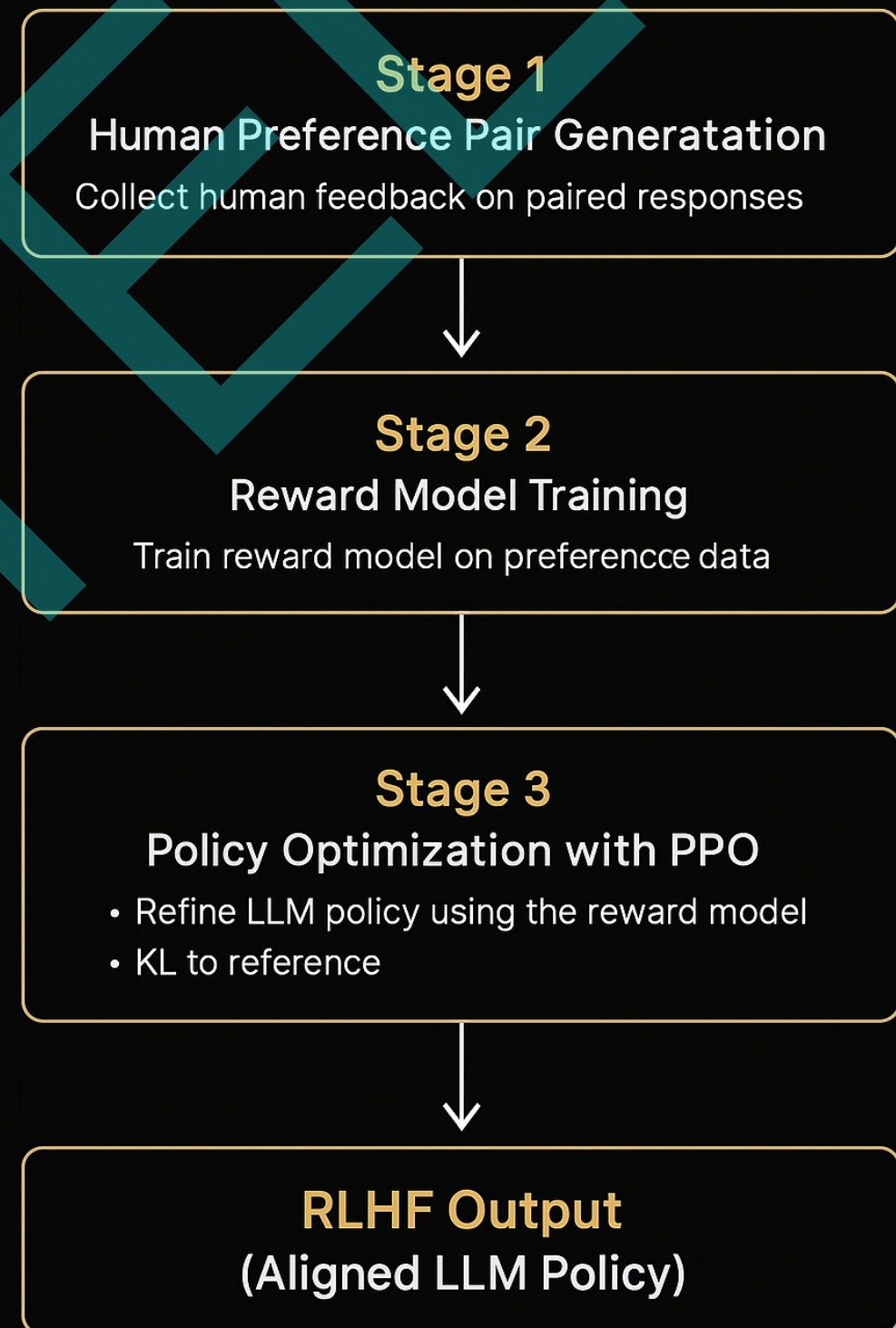
Humans say: B is better than A for a 7-year-old.

RL with PPO

- ✓ We now train a reward model $r_\phi(x, y)$
 - * Let, $r_\phi(x, A) = r_A$ and $r_\phi(x, B) = r_B$
 - * Typical RLHF uses a logistic model of preference:
 - * $P(B \succ A \mid x) = \sigma(r_B - r_A)$ where $\sigma(z) = \frac{1}{1 + e^{-z}}$
 - * The reward model parameters ϕ are optimized using the paired loss.
- ✓ Update the LLM model parameters Θ using the joint loss from the reward model as well as the loss to reduce the model drift.

RL with PPO - Summary

The different stages of
PPO based RLHF



RL with PPO in action

Model	Usage of PPO	Notes
GPT-4 / GPT-4.1 / GPT-4o	✓ Yes (core RLHF)	OpenAI continues PPO-based RLHF at scale.
GPT-3.5 (InstructGPT)	✓ Yes	The original demonstration of PPO-based RLHF.
Anthropic Claude (Claude 1–3)	✓ Yes (Constitutional AI uses RL style; PPO variant)	Constitutional AI uses an RL loop with supervised + RL components.
PaLM, PaLM-2 (Google)	✓ Yes	Google DeepMind uses PPO-based RLHF.
LLaMA-2 Chat (Meta)	✓ Yes	Meta publicly states they trained LLaMA-2-Chat with standard RLHF using PPO.

Direct Policy Optimization

Direct Preference Optimization: Your Language Model is Secretly a Reward Model

Rafael Rafailov^{*†}

Archit Sharma^{*†}

Eric Mitchell^{*†}

Stefano Ermon^{†‡}

Christopher D. Manning[†]

Chelsea Finn[†]

[†]Stanford University [‡]CZ Biohub

{rafailov,architsh,eric.mitchell}@cs.stanford.edu

Direct Preference Optimization

✓ Compared to PPO

* Avoid training of the reward model

- Removes the reward model in the loop

* Directly ingest the human preference data to tune the LLM.

- No sampling loop

- No RL algorithm

- Directly optimize for a supervised loss

DPO

✓ Assume that human pair-wise preference data is available.

$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma \left[(\log \pi_{\theta}(y^{+} | x) - \log \pi_{\theta}(y^{-} | x) - \log \pi_{\text{ref}}(y^{+} | x) + \log \pi_{\text{ref}}(y^{-} | x)) \right]$$

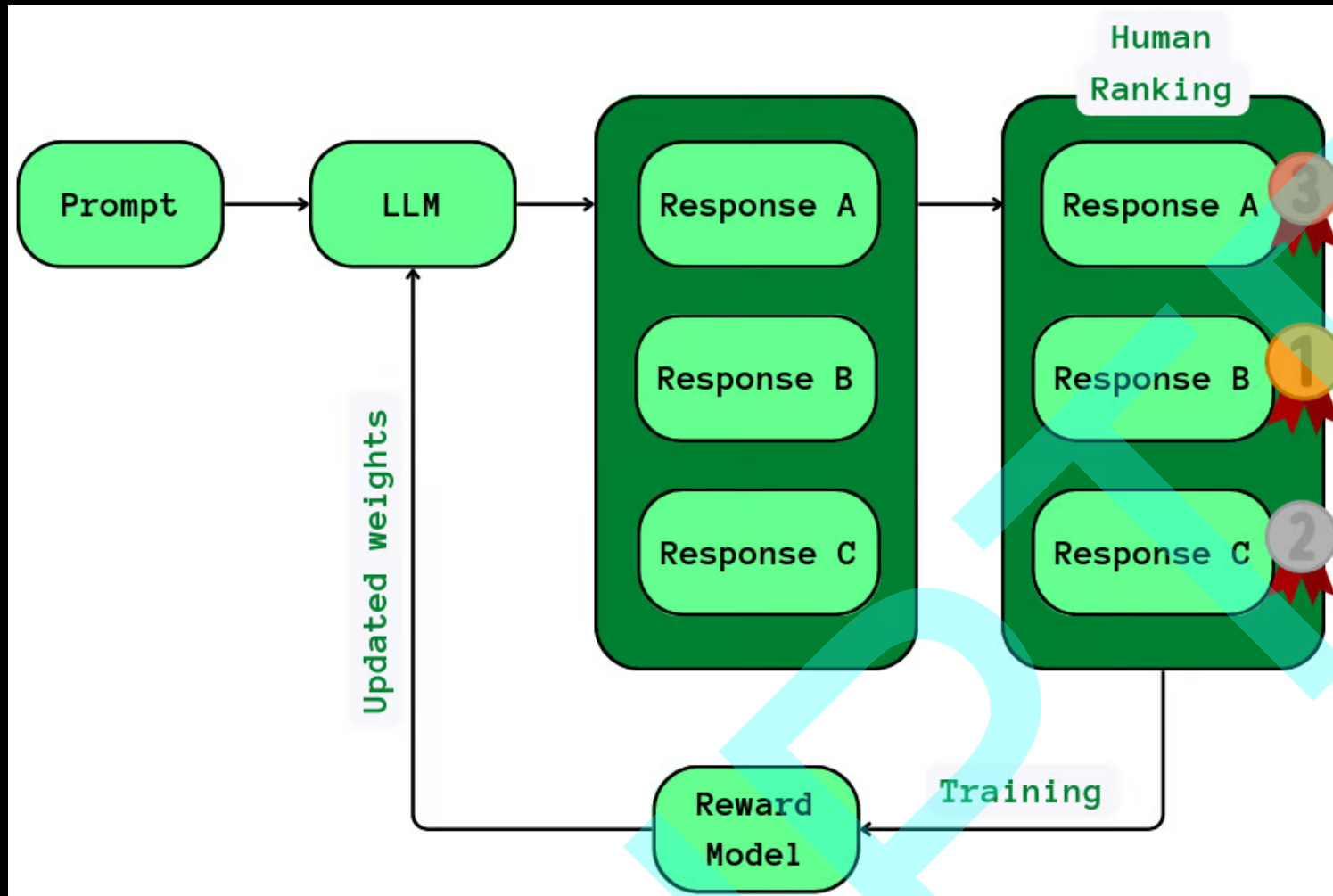
* Here, y^{+} denotes the positive preference output and y^{-} denotes the negative (less) preferred output.

* The model π_{ref} can be the supervised fine-tuned (SFT) model.

* The loss function

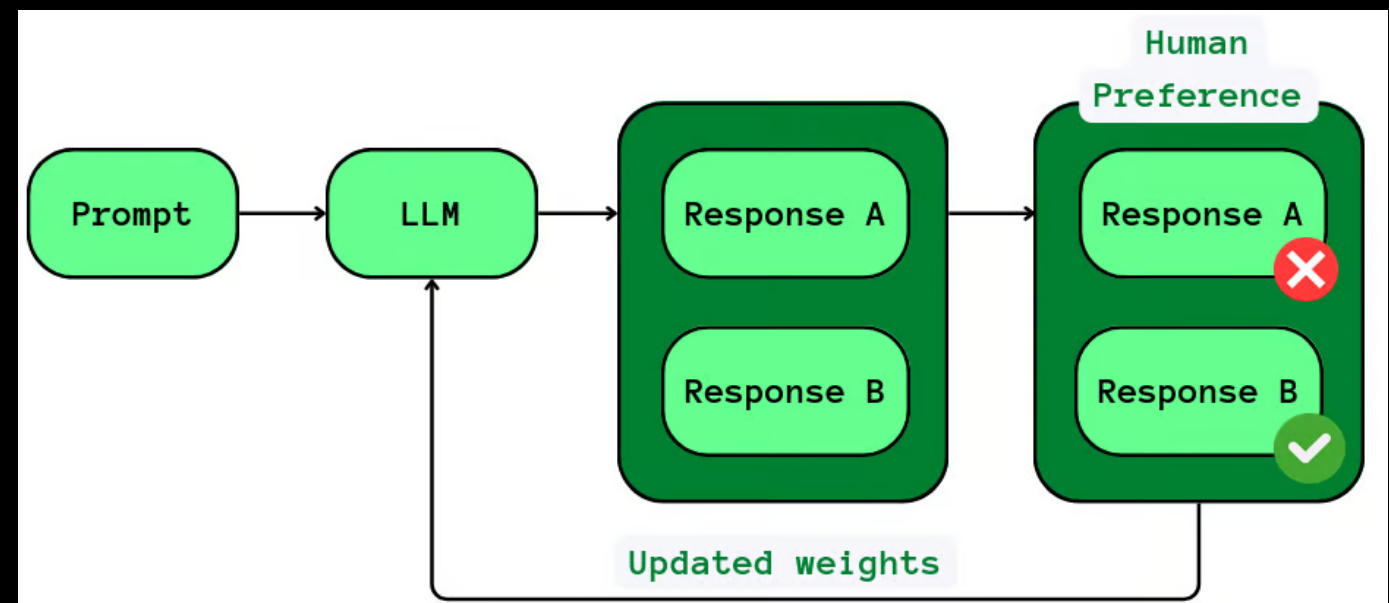
* “**pushes up**” log-prob of preferred outputs and “**pushes down**” log-prob of rejected ones.

PPO versus DPO



PPO

DPO



PPO versus DPO

Feature	PPO	DPO
Uses reward model	✓ Yes	✗ No
Uses RL	✓ Yes	✗ No
Update type	Policy gradient (PPO)	Supervised log-ratio loss
Stability	Medium	High
Cost	High	Low
Data	Rewards	Preference pairs
Good for	Fine control, long-horizon	Alignment, simplicity, speed

DPO in action

Model	Usage	Notes
Mistral Instruct / Mixtral Instruct	✓ DPO used in fine-tuning phases	Mistral widely adopted preference-based methods.
Qwen2.5-Instruct	✓ Uses DPO-style preference tuning	Alibaba mentions preference optimization approaches.
Gemma-Instruct (Google)	✓ Uses DPO-like preference optimization in open-source training	Google explicitly mentions simplified preference training.
Phi-3 / Phi-4 Instruction models	✓ Uses preference optimization steps similar to DPO	Microsoft moves away from heavy RLHF.

Other variants

- ✓ Odds ratio based preference optimization (ORPO)
- ✓ RLAIIF (RL from AI Feedback)
 - * A PPO-like stage but with an AI reward model instead of human feedback. Used in: Claude 3, GPT-4.x research pipelines
- ✓ Group relative policy optimization (GRPO)
 - More recent efforts

What have covered thus far

Transformers
+
Self-supervision

Lecture-1

LLMs
Pre-training and
Post-training

Lecture-2

Benchmarking
LLMs
+
Diffusion

Lecture-3*



Thank You

NOTES